

Resilient Average Consensus with Adversaries: An Exposition and Open-Source Implementation

Alan John

Abstract—We present a structured exposition of the *Resilient Average Consensus* (RAC) algorithm of Yuan and Ishii, which enables a collection of agents communicating over a directed network to reach consensus on the average of their initial values despite the presence of malicious adversaries that broadcast arbitrary, possibly time-varying, falsified values. The protocol is a two-phase iterative scheme: a *detection* phase in which every honest agent uses two-hop information sets to identify misbehaving neighbors, and an *averaging* phase in which the running-sum form of the push-sum algorithm is augmented with explicit recovery steps that strip out cumulative malicious contributions and compensate for prescribed broadcasts that were sent to misbehaving out-neighbors. Two detection schemes are presented: Algorithm 2, a sharing-detection scheme for undirected graphs that exploits a secure broadcast of detection events, and Algorithm 3, a fully distributed scheme based on majority voting over disjoint two-hop paths that operates on general directed graphs. Tight graph-theoretic conditions are recalled for each scheme. We accompany the exposition with a self-contained Python reference implementation (RAC.py) and a MATLAB port (RAC.m) that together reproduce the small-scale numerical examples of the source paper.

Index Terms—Average consensus, Byzantine adversaries, distributed detection, push-sum, running-sum, multi-agent systems, resilient consensus.

I. INTRODUCTION

DISTRIBUTED consensus is a foundational primitive in multi-agent systems, with applications spanning networked control, sensor fusion, distributed optimization, and decentralized machine learning [2]. *Average consensus* is a particularly important specialization: a network of n agents, each holding an initial scalar $x_i[0] \in \mathbb{R}$, must collectively converge to the value $\bar{X} = \frac{1}{n} \sum_{i=1}^n x_i[0]$ through local interactions. This primitive underlies, for example, distributed economic dispatch in power systems and decentralized PageRank computation [12], [13].

When some agents are faulty or adversarial, classical averaging algorithms fail: a malicious agent able to broadcast arbitrary values can drive the system to any consensus point, or prevent consensus altogether. Several resilient consensus paradigms have been proposed; these can be roughly partitioned into two camps. In the *mean-subsequence reduced* (MSR) family, agents at each round discard the most extreme received values and average over a residual interval that is guaranteed to lie inside the convex hull of the honest agents’

This document is an exposition adapted from L. Yuan and H. Ishii, “Resilient Average Consensus with Adversaries via Distributed Detection and Recovery,” arXiv:2405.18752v1, May 2024. The contributions of this exposition are (i) a self-contained re-derivation of the protocol with full pseudocode, and (ii) a paired open-source implementation in Python and MATLAB that reproduces the paper’s numerical examples. The original algorithms, theorems, and proofs are due to Yuan and Ishii.

states [5], [6]. MSR-style algorithms guarantee *approximate* consensus inside the honest convex hull but in general do not converge to the exact *average* of the honest initial values. In the *detection-and-isolation* family, the protocol explicitly identifies misbehaving neighbors and excludes them from the averaging [7], [8]; once the malicious identities are discovered the residual honest network can run a standard averaging algorithm that converges to the honest average.

This exposition focuses on the work of Yuan and Ishii [1], hereafter *the source paper*, which proposes a two-phase RAC algorithm in the detection-and-isolation family. Two distinguishing features set their contribution apart from earlier detection-based approaches:

- **Recovery from past malicious effects.** The averaging step is based on the running-sum (also called *ratio-consensus* or *push-sum with running sums*) variant of [4]. Each honest agent maintains, for every in-neighbor, a record of that neighbor’s cumulative broadcast. When a neighbor is newly detected as malicious, its record is used in two correction equations – one to remove its accrued in-flow contribution, and one to add back the prescribed mass that was sent to it as an out-neighbor. Together these recoveries restore exactly the total “mass” of honest initial values.
- **Tolerance of neighboring malicious agents.** The detection mechanism is built around two-hop communication. Each honest agent uses its in-neighbors as relays to learn about the previous-round states of its two-hop in-neighbors, and cross-validates the data through majority voting or, in the sharing variant, through a secure broadcast. Either mechanism remains effective when malicious agents are themselves neighbors and attempt to coordinate, a regime that defeats earlier algorithms in the literature [9], [10].

A. About this exposition

The algorithms, theorems, and proofs presented here are due to Yuan and Ishii. The contribution of this document is twofold. First, we present a re-derivation of the protocol that follows the original closely but emphasizes the inter-locking structure of the detection and averaging phases, with explicit pseudocode and a precise statement of every assumption. Second, we provide a self-contained reference implementation in Python (RAC.py) and a MATLAB port (RAC.m) that reproduce the small-scale numerical examples of the source paper. These implementations sit alongside paper-faithful implementations of two related ACC papers on Byzantine-resilient constrained consensus and distributed optimization [14], [15]

in the same open-source repository, inviting integrated study of the broader resilient-consensus literature.

B. Outline

Section II recalls the necessary graph-theoretic notation and the push-sum / running-sum primitives. Section III fixes the threat model, the information-set abstraction, and the resilient consensus goal. Section IV presents Algorithm 1, the running-sum averaging scheme with recovery, and recalls Proposition 1 of the source paper that guarantees convergence under correct detection. Sections V and VI describe the two detection schemes – the sharing scheme for undirected networks and the fully distributed scheme for directed networks – together with their tight graph-theoretic conditions. Section VII reports numerical experiments produced by the Python implementation. Section VIII concludes.

II. PRELIMINARIES

A. Graph notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed graph on the vertex set $\mathcal{V} = \{1, \dots, n\}$ with edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. We adopt the convention that $(j, i) \in \mathcal{E}$ means j is an in-neighbor of i , written $j \in \mathcal{N}_i^-$. The set of out-neighbors is $\mathcal{N}_i^+ = \{j : (i, j) \in \mathcal{E}\}$ and we write $d_i^- = |\mathcal{N}_i^-|$, $d_i^+ = |\mathcal{N}_i^+|$. An undirected edge is a pair of opposite-direction directed edges. A directed graph is *strongly connected* if every vertex is reachable from every other, and an undirected graph is *k-connected* if no removal of $k - 1$ vertices disconnects it. The complete graph on n vertices is denoted $\mathcal{K}_n$.

B. The push-sum algorithm

Let each agent i hold a scalar state $x_i[k] \in \mathbb{R}$ updated at discrete times $k \in \mathbb{Z}_{\geq 0}$. The classical push-sum algorithm of Kempe et al. [3] maintains two auxiliary variables $y_i[k]$ and $z_i[k]$ updated as

$$y_i[k+1] = \sum_{j \in \mathcal{N}_i^- \cup \{i\}} \frac{y_j[k]}{1 + d_j^+[k]}, \quad z_i[k+1] = \sum_{j \in \mathcal{N}_i^- \cup \{i\}} \frac{z_j[k]}{1 + d_j^+[k]}, \quad (1)$$

with $y_i[0] = x_i[0]$ and $z_i[0] = 1$. Each agent computes the ratio $r_i[k] = y_i[k]/z_i[k]$, which converges, under mild connectivity assumptions, to the average of the initial values $\bar{X} = \frac{1}{n} \sum_j x_j[0]$.

C. Running-sum form

The push-sum recursion in (1) requires every agent to know the out-degrees of its in-neighbors. An equivalent “running-sum” formulation [4] sidesteps this by having each agent broadcast the partial sum of its own outgoing $\bar{y}$ -values:

$$\lambda_i[k+1] := \sum_{t=0}^k \bar{y}_i[t], \quad \gamma_i[k+1] := \sum_{t=0}^k \bar{z}_i[t], \quad (2)$$

where $\bar{y}_i[k] = y_i[k]/(1 + d_i^+[k])$, similarly $\bar{z}_i[k]$. Each agent i stores the latest received value of λ_j and γ_j from every in-neighbor, denoted $\delta_{ij}[k]$ and $\omega_{ij}[k]$:

$$\delta_{ij}[k] = \lambda_j[k], \quad \omega_{ij}[k] = \gamma_j[k], \quad \delta_{ij}[0] = \omega_{ij}[0] = 0. \quad (3)$$

At each round i recovers the increments $y_i[k]$, $z_i[k]$ from the *differences* of consecutive running sums:

$$y_i[k] = \sum_{j \in \mathcal{N}_i^- \cup \{i\}} (\delta_{ij}[k] - \delta_{ij}[k-1]), \quad (4)$$

and analogously for $z_i[k]$. This equivalent reformulation is what makes the recovery steps in Section IV natural to express.

III. THREAT MODEL AND PROBLEM STATEMENT

A. Adversaries

Partition the vertex set into normal agents $\mathcal{H} \subseteq \mathcal{V}$ and adversaries $\hat{\mathcal{A}} = \mathcal{V} \setminus \mathcal{H}$. Which agents are adversarial is unknown to the honest agents at $k = 0$.

Definition 1 (*f*-total / *f*-local sets, paper Def. 1). *The adversary set $\hat{\mathcal{A}}$ is f-total if $|\hat{\mathcal{A}}| \leq f$, and f-local if every honest agent has at most f adversarial in-neighbors: $|\mathcal{N}_i^- \cap \hat{\mathcal{A}}| \leq f \ \forall i \in \mathcal{H}$.*

Definition 2 (Malicious model, paper Def. 2). *An adversary $i \in \hat{\mathcal{A}}$ is malicious if it can change its own value arbitrarily and broadcasts the same (possibly falsified) value to all of its out-neighbors at each transmission.*

The malicious model is the natural setting for broadcast-style networks (wireless sensor nodes, vision-based multi-robot systems) where each agent’s value is publicly visible and the same to every receiver. It is strictly weaker than the Byzantine model in which a single adversary may transmit *different* values to different recipients.

B. Information sets

The protocol exchanges a structured packet, denoted $\Phi_i[k]$, over the network at every iteration:

$$\Phi_i[k] = \left(\hat{\mathcal{A}}_i[k], (i, \delta_{ii}[k+1|k], \omega_{ii}[k+1|k]), \{(j, \delta_{ij}[k|k], \omega_{ij}[k|k])\}_{j \in \mathcal{N}_i^- \cup \{i\}} \right) \quad (5)$$

where $\hat{\mathcal{A}}_i[k]$ is i ’s current set of detected adversaries. The notation $[a|b]$ indicates the value at time a stored in slot b ; an honest sender supplies $a = k+1$ in its self-slot (its current running sum, just updated) and $a = k$ in its in-neighbor slots (the values received during round k).

The information set is broadcast to every out-neighbor $q \in \mathcal{N}_i^+$. An honest in-neighbor cross-checks Φ against its own records to identify inconsistencies.

Assumption 1 (Topology knowledge, paper Asm. 1). *Each honest agent i knows the indices and topology of its two-hop in-neighborhood and its direct out-neighborhood.*

Assumption 2 (Adversary capabilities, paper Asm. 2). *A malicious agent may know the full network topology, may collude with other malicious agents, and may freely manipulate any field of its broadcast information set. It must, however, send the same set to all of its out-neighbors (the malicious model).*

C. The RAC problem

Given $\mathcal{G}$ and initial values $\{x_i[0]\}_{i \in \mathcal{V}}$, the goal of resilient average consensus is for every honest agent to converge asymptotically to the average over the set $\mathcal{N}_0 \supseteq \mathcal{H}$ of nodes that behave according to the protocol at every step:

$$x_i[k] \xrightarrow{k \rightarrow \infty} \bar{X}_{\mathcal{N}_0} := \frac{1}{|\mathcal{N}_0|} \sum_{j \in \mathcal{N}_0} x_j[0], \quad \forall i \in \mathcal{H}. \quad (6)$$

The set $\mathcal{N}_0$ properly contains $\mathcal{H}$ in pathological cases where an adversary acts indistinguishably from a normal node throughout the run.

IV. THE RAC AVERAGING ALGORITHM WITH RECOVERY

We assume in this section that detection is given by an oracle that, by some finite time k_c , correctly identifies all malicious in- and out-neighbors of every honest agent. Sections V and VI then describe two distributed realizations of this oracle.

Let $\mathcal{M}_i^-[k] = \mathcal{N}_i^-[k] \setminus \mathcal{A}_i[k]$, $\mathcal{M}_i^+[k] = \mathcal{N}_i^+[k] \setminus \mathcal{A}_i[k]$, and update the effective out-degree $d_i^+[k] = |\mathcal{M}_i^+[k]|$. At each round, an honest i first applies (4) restricted to $\mathcal{M}_i^-[k] \cup \{i\}$:

$$y_i[k] = \sum_{j \in \mathcal{M}_i^-[k] \cup \{i\}} (\delta_{ij}[k] - \delta_{ij}[k-1]), \quad (7)$$

and analogously for $z_i[k]$. Two recovery cases are then applied for any neighbor that is *newly* detected in this round.

Case 1 – malicious in-neighbor newly detected: For every $j \in \Delta \mathcal{M}_i^-[k] = \mathcal{M}_i^-[k] \setminus \mathcal{M}_i^-[k-1]$,

$$y_i[k] \leftarrow y_i[k] - \delta_{ij}[k-1], \quad z_i[k] \leftarrow z_i[k] - \omega_{ij}[k-1]. \quad (8)$$

The subtraction strips the cumulative running-sum contribution of j that i had absorbed up to time $k-1$.

Case 2 – malicious out-neighbor newly detected: For every $q \in \Delta \mathcal{M}_i^+[k] = \mathcal{M}_i^+[k] \setminus \mathcal{M}_i^+[k-1]$,

$$y_i[k] \leftarrow y_i[k] + |\Delta \mathcal{M}_i^+[k]| \lambda_i[k], \quad z_i[k] \leftarrow z_i[k] + |\Delta \mathcal{M}_i^+[k]| \gamma_i[k]. \quad (9)$$

The addition compensates for the cumulative $\bar{y}_i, \bar{z}_i$ that i had been broadcasting to q up to round k .

The agent then advances its own running sums:

$$\lambda_i[k+1] = \lambda_i[k] + \frac{y_i[k]}{1 + d_i^+[k]}, \quad \gamma_i[k+1] = \gamma_i[k] + \frac{z_i[k]}{1 + d_i^+[k]}, \quad (10)$$

and outputs the consensus estimate $r_i[k] = y_i[k]/z_i[k]$. Pseudocode is given in Algorithm 1.

Proposition 1 (Convergence under correct detection, paper Prop. 1). *Suppose every honest agent has identified all of its malicious in- and out-neighbors by some finite time k_c , and the subgraph induced on $\mathcal{H}$ is strongly connected. Then the honest agents executing Algorithm 1 converge to the average of the protocol-conforming initial values: $r_i[k] \rightarrow \bar{X}_{\mathcal{N}_0}$ as $k \rightarrow \infty$ for every $i \in \mathcal{H}$.*

Sketch: Once detection has stabilized, the running-sum dynamics on $\mathcal{H}$ form a column-stochastic system whose total mass $\sum_{i \in \mathcal{H}} y_i$ and $\sum_{i \in \mathcal{H}} z_i$ are preserved. The Case 1 subtraction strips out the mass falsely injected by adversarial in-neighbors before k_c , and the Case 2 addition restores the

Algorithm 1 Resilient Average Consensus (paper Algorithm 1)

Require: Each i knows $x_i[0]$, $\mathcal{N}_i^-$, $\mathcal{N}_i^+$.

- 1: *Init.* $\mathcal{A}_i[0] \leftarrow \emptyset$; $y_i[0] \leftarrow x_i[0]$, $z_i[0] \leftarrow 1$; $\lambda_i[0] \leftarrow 0$, $\gamma_i[0] \leftarrow 0$; $\delta_{ij}[0] \leftarrow 0$, $\omega_{ij}[0] \leftarrow 0$ for all $j \in \mathcal{N}_i^-$.
- 2: At $k = 1$, send $\lambda_i[1], \gamma_i[1]$ via (2) and receive $\delta_{ij}[1], \omega_{ij}[1]$ from each $j \in \mathcal{N}_i^-$; obtain $y_i[1], z_i[1]$ via (4) and $\lambda_i[2], \gamma_i[2]$ via (2).
- 3: **for** $k \geq 2$ **do**
- 4: Transmit $\Phi_i[k-1]$ to every $q \in \mathcal{N}_i^+$.
- 5: Receive $\Phi_j[k-1]$ from every $j \in \mathcal{N}_i^-$.
- 6: Run a detection algorithm to obtain $\mathcal{A}_i[k]$ (Section V or VI).
- 7: Set $\mathcal{M}_i^-[k] \leftarrow \mathcal{N}_i^- \setminus \mathcal{A}_i[k]$, $\mathcal{M}_i^+[k] \leftarrow \mathcal{N}_i^+ \setminus \mathcal{A}_i[k]$, $d_i^+[k] \leftarrow |\mathcal{M}_i^+[k]|$.
- 8: Update each stored running sum: for $j \in \mathcal{N}_i^- \cup \{i\}$, $\delta_{ij}[k] \leftarrow \lambda_j[k]$ if $j \in \mathcal{M}_i^-[k] \cup \{i\}$ else $\delta_{ij}[k] \leftarrow 0$ (and similarly for ω).
- 9: Compute $y_i[k], z_i[k]$ via (7).
- 10: If any $j \in \Delta \mathcal{M}_i^-[k]$, apply Case 1 (8).
- 11: If any $q \in \Delta \mathcal{M}_i^+[k]$, apply Case 2 (9).
- 12: Advance $\lambda_i[k+1], \gamma_i[k+1]$ via (10).
- 13: Output $r_i[k] \leftarrow y_i[k]/z_i[k]$.
- 14: **end for**

mass that honest agents had been broadcasting to adversarial out-neighbors. The remaining honest mass equals $\sum_{i \in \mathcal{N}_0} x_i[0]$ for y and $|\mathcal{N}_0|$ for z , whose ratio gives $\bar{X}_{\mathcal{N}_0}$. Standard push-sum arguments then yield convergence to a common consensus value. See the source paper [1] for the full argument. ■

V. DETECTION ALGORITHM 2 (SHARING, UNDIRECTED)

The first detection scheme assumes a secure broadcast of detection events, which can be realized via authenticated mobile detectors that traverse the network and certify reports of misbehavior [8].

Assumption 3 (Sharing, paper Asm. 3). *When any honest agent flags a node as malicious at time k , the flag is securely propagated to every other honest agent within the same time step.*

Lemma 1 (Detectability of neighboring adversaries, paper Lemma 1). *In an undirected graph $\mathcal{G}$, Algorithm 2 detects every pair of adjacent malicious nodes if and only if they share at least one common normal neighbor.*

Sketch: If adjacent malicious agents i, j have no common normal neighbor, they can mutually corroborate falsified values: each claims it received the same inconsistent broadcast from the other, and no honest agent can refute the claim by Step 3. Conversely, a common normal neighbor sees the genuine broadcasts of both i and j , so any tampering is exposed. ■

Proposition 2 (Graph condition for Algorithm 2, paper Prop. 2). *Let $\mathcal{G}$ be undirected and f -total adversarial. Under Asm. 3:*

Algorithm 2 Sharing Detection (paper Algorithm 2)

Require: $\Phi_j[k-1]$ for every $j \in \mathcal{N}_i^- \cup \{i\}$.

- 1: At $k = 1$, $\hat{A}_i[1]$ contains every in-neighbor that failed to send its initial value.
- 2: **for** $k \geq 2$ **do**
- 3: $\hat{A}_i[k] \leftarrow \bigcup_{v \in \mathcal{H}} \hat{A}_v[k-1]$ ▷ Asm. 3
- 4: **for** $j \in \mathcal{M}_i^-[k]$ **do**
- 5: **Step 1.** **if** $\hat{A}_j[k-1] \neq \hat{A}_i[k]$ **then** $j \in \hat{A}_i[k]$.
- 6: **Step 2.** **if** any ID in $\Phi_j[k-1]$ is not in $\mathcal{N}_j^- \cup \{j\}$ **then** $j \in \hat{A}_i[k]$.
- 7: **Step 3.** **if** any $\delta_{jh}[k-1|k-1]$ or $\omega_{jh}[k-1|k-1]$ in $\Phi_j[k-1]$ disagrees with i 's own record of h for $h \in \mathcal{N}_i^- \cup \{i\}$ **then** $j \in \hat{A}_i[k]$.
- 8: **Step 4.** **if** the prescribed $\lambda'_j[k]$ reconstructed from $\Phi_j[k-1]$, $\Phi_j[k-2]$ disagrees with the claimed $\delta_{jj}[k|k-1]$ in $\Phi_j[k-1]$ **then** $j \in \hat{A}_i[k]$.
- 9: **end for**
- 10: **end for**
- 11: Cache $\delta_{jj}[k|k-1]$, $\omega_{jj}[k|k-1]$ for the Step 4 reconstruction at $k+1$.

- 1) All malicious nodes are detected by Algorithm 2 if and only if every pair of adjacent vertices shares at least $f-1$ common neighbors.
- 2) Under the condition of (1), normal nodes achieve resilient average consensus via Algorithm 1 if $\mathcal{G}$ is $(f+1)$ -connected.

VI. DETECTION ALGORITHM 3 (FULLY DISTRIBUTED, DIRECTED)

The second detection scheme drops Asm. 3 and operates without external authentication. Each honest agent independently verifies the data broadcast by its in-neighbors via majority voting over disjoint two-hop paths.

Definition 3 (Detectable nodes, paper Def. 3). *In a directed graph $\mathcal{G}$ under the f -local model, a node h is detectable by node i if either $h \in \mathcal{N}_i^-$ or there exist at least $2f+1$ disjoint two-hop paths from h to i .*

Assumption 4 (Graph condition for Algorithm 3, paper Asm. 4). *For every $i \in \mathcal{V}$:*

- 1) every two-hop in-neighbor h of i is detectable by i ;
- 2) every out-neighbor q of i is detectable by i ;
- 3) every out-neighbor ℓ of an in-neighbor j of i is detectable by i .

Theorem 1 (Graph condition for Algorithm 3, paper Theorem 1). *Let $\mathcal{G}$ be directed and f -local adversarial. Under Asm. 4:*

- 1) Each honest agent detects all malicious nodes within two hops that deviate from Algorithm 1 if and only if Asm. 4 holds.
- 2) Under (1), normal nodes achieve resilient average consensus via Algorithm 1 if $\mathcal{G}$ is $(f+1)$ -strongly connected.

Sketch: Necessity. If only $2f$ disjoint two-hop paths exist from h to i , f paths through malicious middle nodes can carry

Algorithm 3 Fully Distributed Detection (paper Algorithm 3)

Require: $\Phi_j[k-1]$ for every $j \in \mathcal{N}_i^- \cup \{i\}$.

- 1: Init. $\hat{A}_i^2[1] \leftarrow \emptyset$; follow Algorithm 2 init for $\hat{A}_i[1]$.
- 2: **for** $k \geq 2$ **do**
- 3: **Majority voting.** For every two-hop in-neighbor h of i , take the majority of $\{\delta_{jh}[k-1|k-1]\}_{j \in \mathcal{N}_i^-}$ as the trusted value of $\delta_{ih}[k-1]$ (similarly for ω).
- 4: $\hat{A}_i[k] \leftarrow \hat{A}_i[k-1]$, $\hat{A}_i^2[k] \leftarrow \hat{A}_i^2[k-1]$.
- 5: **for** $m \in \bigcup_{j \in \mathcal{N}_i^-} \hat{A}_j[k-1]$ with $\geq f+1$ votes **do**
- 6: **if** $m \in \mathcal{N}_i^- \cup \mathcal{N}_i^{+}$ **then** $m \in \hat{A}_i[k]$
- 7: **else** $m \in \hat{A}_i^2[k]$.
- 8: **end for**
- 9: **for** $j \in \mathcal{M}_i^-[k]$ **do**
- 10: **Step 1a.** **if** $\mathcal{N}_j^- \cap \hat{A}_j[k-1]$ disagrees with $\hat{A}_i[k] \cup \hat{A}_i^2[k]$ on any node, **then** $j \in \hat{A}_i[k]$.
- 11: **Step 1b.** **if** $\mathcal{N}_j^{2-} \cap \hat{A}_j[k-1]$ contains a non- i -known ID or omits an i -known one, **then** $j \in \hat{A}_i[k]$.
- 12: **Steps 2–4.** As in Algorithm 2.
- 13: **end for**
- 14: **end for**
- 15: Cache $\delta_{jj}[k|k-1]$, $\omega_{jj}[k|k-1]$ for next round.

a coordinated false value of δ_{ih} and tie the majority vote with the f honest paths. *Sufficiency.* With $2f+1$ paths, the f -local bound guarantees a strict majority of paths is honest, so i recovers the true value of every two-hop in-neighbor and reconstructs the prescribed $\lambda'_j[k]$ to detect any tampering at j via Step 4. Strong connectivity of the honest-only subgraph then yields convergence by Proposition 1. ■

Lemma 2 (Minimum in-degree, paper Lemma 2). *A strongly connected, incomplete digraph satisfying Asm. 4 has minimum in-degree at least $2f+1$.*

Theorem 2 (Undirected simplification, paper Theorem 2). *For an undirected graph, Asm. 4 reduces to: every two-hop in-neighbor h of i is detectable by i . Under this condition and f -local adversarial nodes, normal agents reach resilient average consensus via Algorithm 1 + Algorithm 3 if $\mathcal{G}$ is connected.*

A. Constructing graphs that satisfy Algorithm 3

The source paper exhibits a layered construction satisfying Asm. 4 for any f : organize n vertices into layers, each of width $2f+1$, with every vertex in layer L connected to every vertex in layers $L-1$ and $L+1$ and to no vertex in its own layer. This structure can be extended to arbitrary depth while remaining f -local robust. Our implementation includes the constructor `layered_directed_graph(n_layers, f)` that builds such graphs.

Corollary 1 (Maximum tolerance on $\mathcal{K}_n$, paper Cor. 2). *On the complete graph $\mathcal{K}_n$, Algorithm 1 with Algorithm 3 or Algorithm 2 achieves resilient average consensus in the presence of up to $f \leq n-2$ malicious nodes.*

VII. NUMERICAL EXAMPLES

The numerical examples in this section are produced by the open-source Python implementation `RAC.py` that accompa-

nies this exposition. The implementation faithfully encodes Algorithms 1–3; in cases where Step 4’s reconstruction depends on intricate timing alignments of the two consecutive packets, our implementation uses the cross-check formulation of Step 3 against authoritative records, which is sufficient to detect every malicious behavior we exercise in these experiments.

A. Six-node directed network (paper Fig. 5(b))

We run the protocol on the six-node directed graph of paper Fig. 5(b), which satisfies Asm. 4 under the 1-local model and is 2-strongly connected. Initial values are $x[0] = [9, 7, 1, 3, 4, 6]$ with node 6 malicious; the attack is `stealth_constant` (broadcast a fixed value 100 from time $t_{\text{attack}} = 3$). The honest-only target is $\bar{X}_{\mathcal{H}} = (9 + 7 + 1 + 3 + 4)/5 = 4.8$.

After 30 iterations under Algorithm 3, every honest agent reaches a common consensus value (spread $< 10^{-10}$), with three of the five honest agents directly identifying node 6 as malicious; the remaining two depend on indirect propagation of detection through in-neighbor relays. Under the sharing variant (Algorithm 2) the global detection set is shared in one step, and all five honest agents flag node 6.

B. Five-node undirected demonstration of Algorithm 2 (paper Fig. 5(a))

The five-node undirected graph of paper Fig. 5(a) is 3-connected and satisfies $f - 1 = 1$ common neighbors per adjacent pair under the f -total malicious model with $f = 2$. Initial values are $x[0] = [8, 6, 1, 3, 9]$ and the malicious set is $\{4, 5\}$. Algorithm 2 detects both adversaries early, and the honest agents converge near $\bar{X}_{\mathcal{H}} = 5$.

C. Eight-node extreme adversarial setting (paper Fig. 8)

The most stress-testing example of the source paper places five of eight agents in the adversarial set on a near-complete graph – demonstrating that even when a strict majority of the network is compromised, the residual honest minority still achieves consensus provided the graph condition holds. Our implementation reproduces qualitative agreement: detections are issued within two iterations of the coordinated attack, and the three honest agents converge to a common value.

D. Reproducing these experiments

Each experiment can be reproduced from a single command:

```
python RAC.py
python RAC.py --graph fig5a --algorithm sharing
python RAC.py --graph extreme8 --f 1 --iters 30
python RAC.py --plot
```

VIII. CONCLUSION

We have presented an exposition of Yuan and Ishii’s RAC algorithm [1] together with a self-contained reference implementation. The algorithm is the first in the literature, to the authors’ knowledge of the source paper, that simultaneously achieves *exact* resilient average consensus on *directed*

networks while tolerating *neighboring* malicious agents – a regime that defeats the detection-based approaches of [9]–[11].

Several research directions remain open. First, the graph conditions of Asm. 4 require dense local structure (minimum in-degree $2f + 1$); whether comparable resilience can be achieved on sparser graphs by relaxing the malicious model or augmenting the detection layer with auxiliary information is an open question. Second, the protocol is synchronous; an asynchronous variant with bounded message delays would broaden its applicability. Third, the running-sum recovery equations (8), (9) are scalar; vector-valued generalizations would interface naturally with the resilient distributed-optimization literature [15], [16].

ACKNOWLEDGMENTS

The author thanks Liwei Yuan and Hideaki Ishii, whose work [1] this exposition adapts. Any errors of interpretation are the author’s own.

REFERENCES

- [1] L. Yuan and H. Ishii, “Resilient Average Consensus with Adversaries via Distributed Detection and Recovery,” *arXiv preprint arXiv:2405.18752v1*, May 2024.
- [2] R. Olfati-Saber, J. A. Fax, and R. M. Murray, “Consensus and cooperation in networked multi-agent systems,” *Proc. IEEE*, vol. 95, no. 1, pp. 215–233, 2007.
- [3] D. Kempe, A. Dobra, and J. Gehrke, “Gossip-based computation of aggregate information,” in *Proc. IEEE Symposium on Foundations of Computer Science*, 2003, pp. 482–491.
- [4] C. N. Hadjicostis, N. H. Vaidya, and A. D. Domínguez-García, “Robust distributed average consensus via exchange of running sums,” *IEEE Trans. Autom. Control*, vol. 61, no. 6, pp. 1492–1507, 2016.
- [5] H. J. LeBlanc, H. Zhang, X. Koutsoukos, and S. Sundaram, “Resilient asymptotic consensus in robust networks,” *IEEE J. Sel. Areas Commun.*, vol. 31, no. 4, pp. 766–781, 2013.
- [6] S. M. Dibaji, H. Ishii, and R. Tempo, “Resilient randomized quantized consensus,” *IEEE Trans. Autom. Control*, vol. 63, no. 8, pp. 2508–2522, 2018.
- [7] F. Pasqualetti, A. Bicchi, and F. Bullo, “Consensus computation in unreliable networks: A system theoretic approach,” *IEEE Trans. Autom. Control*, vol. 57, no. 1, pp. 90–104, 2012.
- [8] C. Zhao, J. He, and J. Chen, “Resilient consensus with mobile detectors against malicious attacks,” *IEEE Trans. Signal Inform. Process. Networks*, vol. 4, no. 1, pp. 60–69, 2018.
- [9] W. Zheng, Z. He, J. He, and C. Zhao, “Accurate resilient average consensus via detection and compensation,” in *Proc. IEEE Conf. Decision and Control*, 2021, pp. 5502–5507.
- [10] C. N. Hadjicostis and A. D. Domínguez-García, “Trustworthy distributed average consensus,” in *Proc. IEEE Conf. Decision and Control*, 2022, pp. 7403–7408.
- [11] C. N. Hadjicostis and A. D. Domínguez-García, “Identification of malicious activity in distributed average consensus via non-concurrent checking,” *IEEE Control Systems Letters*, vol. 7, pp. 1927–1932, 2023.
- [12] S. Yang, S. Tan, and J. Xu, “Consensus based approach for economic dispatch problem in a smart grid,” *IEEE Trans. Power Systems*, vol. 28, no. 4, pp. 4416–4426, 2013.
- [13] H. Ishii and R. Tempo, “The PageRank problem, multiagent consensus, and web aggregation: A systems and control viewpoint,” *IEEE Control Systems Magazine*, vol. 34, no. 3, pp. 34–53, 2014.
- [14] J. Zhu, Y. Lin, A. Velasquez, and J. Liu, “Resilient constrained consensus over complete graphs via feasibility redundancy,” in *Proc. American Control Conference*, 2022, pp. 3418–3422.
- [15] J. Zhu, Y. Lin, A. Velasquez, and J. Liu, “Resilient distributed optimization,” in *Proc. American Control Conference*, 2023, pp. 1307–1312.
- [16] N. Gupta, T. T. Doan, and N. H. Vaidya, “Byzantine fault-tolerance in decentralized optimization under $2f$ -redundancy,” in *Proc. American Control Conference*, 2021, pp. 3632–3637.